



DON BOSCO INSTITUTE OF TECHNOLOGY

**(An Autonomous Institute, Affiliated to VTU, Belagavi)
Mysuru Road, Kumbalagodu, Bengaluru – 560 074.**

Department of Information Science & Engineering

(UG Program Accredited by NBA 2025-2028)

DATA STRUCTURES LABORATORY MANUAL

III Semester

Course Code: B24ISL38

w.e.f September 2025

Document Log

Name of the document	Data Structures Laboratory Manual
Subject code	B24ISL38
Scheme	2024
Current version number and date	V 1.0 / 05.08.2025
Manual Prepared by	Ms. Smitha B N Dr. Shankar Gowda S R Dr. B K Raghavendra Prof. Dr. N. Guruprasad
Approved by	Dr. B K Raghavendra HOD - Dept. of ISE, DBIT

Table of Contents

Sl. No.	Particulars	Page #
1	Vision and Mission of Department	i
2	PEOs and PSOs	i
3	POs	ii
4	Course details • Course Outcomes • CO-PO-PSO Mapping	iii
5	RUBRICS for CIE and SEE	iv
6	LAB PROGRAMS	
	Program 1 Develop a C program to sort the elements of an array using Radix sort technique.	1
	Program 2 Develop a C Program to implement Hashing using Linear and Quadratic Probing	3
	Program 3 Develop a menu driven program in C to implement primitive operations on stack a) Push b) Pop c) Display. The program should print appropriate messages for stack overflow and stack underflow	8
	Program 4 Develop a C program to convert INFIX notation to POSTFIX notation	15
	Program 5 Develop a C program for evaluation of POSTFIX notation	17
	Program 6 Develop a menu driven program in C language to implement QUEUE data structures that performs following primitive operations: a) insert b) delete and c) display	19
	Program 7 Develop a menu driven program in C language to for CIRCULAR QUEUE that performs following primitive operations: a) insert b) delete and c) display	22
	Program 8 Develop a menu driven program in C to perform primitive operations on single linked list.	30
	Program 9 Develop a C program to perform insertion, deletion, and traversal of a binary search tree	46
	Program 10 Develop a C program to traverse a graph using Breadth First Search	54
7	VIVA QUESTIONS	57

Vision of the Department

Empower the students to thrive for excellence in the area of Information Science and Engineering to use technology for sustainable development for social needs.

Mission of the Department

M1: To provide theoretical and practical knowledge of Information Science and Engineering.

M2: To provide inter disciplinary knowledge to be the leaders with social responsibility.

M3: To collaborate with industry to facilitate content beyond syllabus.

M4: To promote team work through projects, co-curricular and social activities.

Program Educational Objectives (PEOs)

PEO1: Graduates will contribute in development of software applications keeping abreast of the development.

PEO2: Exhibit competence as individual, in teams with leadership and managerial skills.

PEO3: By optimizing the technology, the graduates will be able to adopt lifelong learning.

Program Specific Outcomes (PSOs)

PSO1: Apply the Mathematical tools, Electronics & Embedded Systems Knowledge, and Programming Knowhow to develop software's.

PSO2: Use of Artificial Intelligence and Machine Learning, High-Performance Computing, Cloud Computing, Network Security technologies and Software Engineering for providing solutions to the technological and social needs.

PSO3: Work individually and in teams, ethically exhibiting the managerial and leadership skills with sustainable Environment.

Knowledge and Attitude Profile (WK)	
WK1	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK2	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
WK4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5	Knowledge, including efficient resource use, environmental impacts, whole-life cost, re-use of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
WK7	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development
WK8	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK9	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.

Program Outcomes	
P01	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
P02	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
P03	Design/development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
P04	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
P05	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
P06	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
P07	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
P08	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
P09	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
P010	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
P011	Life-long learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Course Outcomes

At the end of the course, student will be able to

C01	Develop programs related to arrays and hashing
C02	Develop programs related to stacks and its applications
C03	Develop programs related to queues
C04	Demonstrate use of dynamic memory allocation to implement single linked list
C05	Develop programs related to trees
C06	Apply concept of graphs to implement traversal algorithms

CO-PO-PSO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2	PSO3
CO1	3	3	3	-	3	-	3		-	-	-	3	3	-
CO2	3	3	3	-	3	-	3		-	-	-	3	3	-
CO3	3	3	3	-	3	-	3		-	-	-	3	3	-
CO4	3	3	3	-	3	-	3		-	-	-	3	3	-
CO5	3	3	3	-	3	-	3		-	-	-	3	3	-
CO6	3	3	3	-	3	-	3		-	-	-	3	3	-
AVG	3	3	3	-	3	-	3		-	-	-	3	3	-

RUBRICS FOR CIE & SEE

WEEKLY ASSESSMENT – 30 marks

Sl. No	ACTIVITY	MARKS
1	Record +Viva	30

LAB RUBRICS

Rubrics for Evaluation of Lab Record:

Attribute	Max Marks	Good	Satisfactory	Poor
		4-5	1-3	0
Write Up and Time Management	5	- Written complete program without any errors - Written input and expected output for all the test cases. - Written program with proper indentation - Index entry promptly done and submits on time	- Written Program with few logical errors. - Written input and expected output for few test cases. - Written program with moderate indentation	- Not submitted - Input and expected output is not written
	Max Marks	15-20	5-14	0-4
Conduction of experiment	20	All possible cases of input and output demonstrated	The code is not tested with all possible cases.	Unable to execute within the allotted
	Max Marks	4-5	1-3	0
Viva	5	- Able to explain the design of algorithm and logic. - Able to answer all question related to the concept.	- Partially understood the logic of the experiment. - Able to answer 1 out of 3 questions	- Not understood the logic of the experiment. - Not answering any question

INTERNAL ASSESSMENT EVALUATION

(End of Semester)

Sl. No	ACTIVITY	MARKS
1	Write-up	15
2	Conduction	70
3	Viva	15
TOTAL MARKS		100 (scaled down to 20)

Rubrics for Evaluation of Internal Test:

Attribute	Max Marks	Excellent	Good	Satisfactory	Poor
		11-15	6-10	1-5	0
Write-up	15	• Completes source code. • No syntax and logical errors • All possible inputs listed with expected output	• Completes source code. • Syntax and logical errors exist.	• Incomplete source code. • All possible inputs listed with expected output	• Change of program
	Max Marks	60-70	20-59	11-19	0-10
Execution	70	• Able to debug the program and get the right set of outputs • Able to modify the Program as per the examiner's Direction	• Able to execute program for few possible inputs without errors	• Able to clear the errors. • Not able to execute program	• Not able to clear the errors and execute the program
	Max Marks	11-15	6-10	1-5	0
Viva - Voce	15	• Able to answer all questions correctly	• Able to answer 3 questions out of 5 questions correctly	• Able to answer 2 - 5 questions	• Not able to answer any questions.

FINAL INTERNAL ASSESSMENT CALCULATION		
Sl . No	ACTIVITY	MARKS
1	Average of weekly entries	30
2	Internal Assessment	20
	TOTAL	50

Program # 1. Develop a C program to sort the elements of an array using Radix sort technique.

APPLICATIONS / IMPORTANCE:

- 1) **Sorting Large Numbers:** Efficient for sorting large datasets of integers, especially when the range of the numbers is limited.
- 2) **Sorting Fixed-Length Strings:** Useful in sorting strings of the same length, such as sorting postal codes, phone numbers, or other identifiers.
- 3) **Digital Data Processing:** Commonly used in digital signal processing and sorting large arrays of data with fixed-length keys.
- 4) **Library Systems:** Helps in sorting books by ISBN numbers or other fixed-length numeric identifiers.
- 5) **Computer Graphics:** Used in graphics applications for tasks like sorting colors based on their RGB values, where each component can be treated as a digit.
- 6) **Date Sorting:** Efficient for sorting dates stored in a numeric format, such as YYYYMMDD, by processing year, month, and day as separate digits.
- 7) **Financial Systems:** Applied in financial software for sorting transaction IDs, account numbers, or other numeric data where performance is crucial.

SOURCE CODE:

```
#include<stdio.h>
int get_max (int a[], int n){
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}

void radix_sort (int a[], int n){
    int bucket[10][10], bucket_cnt[10];
    int i, j, k, r, NOP = 0, divisor = 1, lar, pass;
    lar = get_max (a, n);
    while (lar > 0){
        NOP++;
        lar /= 10;
    }
    for (pass = 0; pass < NOP; pass++){
        for (i = 0; i < 10; i++){
            bucket_cnt[i] = 0;
        }
    }
}
```

```
    for (i = 0; i < n; i++){
        r = (a[i] / divisor) % 10;
        bucket[r][bucket_cnt[r]] = a[i];
        bucket_cnt[r] += 1;
    }
    i = 0;
    for (k = 0; k < 10; k++){
        for (j = 0; j < bucket_cnt[k]; j++){
            a[i] = bucket[k][j];
            i++;
        }
    }
    divisor *= 10;
    printf ("\n After pass %d : ", pass + 1);
    for (i = 0; i < n; i++)
        printf ("\n %d",a[i]);
    }
}
```

```
int main (){
    int i, n, a[10];
    printf ("\n Enter the number of elements to be sorted : ");
    scanf ("%d", &n);
    printf ("\n Enter the elements : ");
    for (i = 0; i < n; i++){
        scanf ("%d",&a[i]);
    }
    radix_sort (a, n);
    printf ("\n Sorted items : ");
    for (i = 0; i < n; i++)
        printf ("%d",a[i]);
    return 0;
}
```

SAMPLE OUTPUT

```
Enter the number of elements to be sorted :5
Enter the elements : 256
143
761
412
396
```

After pass 1 :

761

412

143

256

396

After pass 2 :

412

143

256

761

396

After pass 3 :

143

256

396

412

761

Sorted elements are :

143

256

396

412

761

Program 2: Develop a C Program to implement Hashing using Linear and Quadratic Probing

APPLICATIONS / IMPORTANCE:

- 1) **Simple implementation:** Linear probing is easy to implement and understand, making it suitable for situations where simplicity is prioritized over performance.
- 2) **Good cache performance:** It exhibits good cache performance due to the sequential nature of its search, which can lead to faster lookups if the table is not heavily loaded.
- 3) **Reduced Clustering:** Quadratic probing significantly reduces the primary clustering problem associated with linear probing.
- 4) **Improved Performance:** It generally offers better performance than linear probing, especially in moderately loaded hash tables, by distributing items more evenly.

SOURCE CODE:

```
#include <stdio.h>
#include <conio.h>
int tsize;

int hasht(int key) {
    int i;
    i = key%tsize ;
    return i;
}

//-----LINEAR PROBING-----
int rehashl(int key) {
    int i;
    i = (key+1)%tsize ;
    return i;
}

//-----QUADRATIC PROBING-----
int rehashq(int key, int j) {
    int i;
    i = (key+(j*j))%tsize ;
    return i;
}

void main() {
    int key,arr[20],hash[20],i,n,s,op,j,k ;
    clrscr() ;
    printf ("Enter the size of the hash table: ");
    scanf ("%d",&tsize);

    printf ("\nEnter the number of elements: ");
    scanf ("%d",&n);

    for (i=0;i<tsize;i++)
        hash[i]=-1 ;

    printf ("Enter Elements: ");
    for (i=0;i<n;i++) {
        scanf ("%d",&arr[i]);
    }

    do {
        printf("\n1.Linear Probing ");
        printf("\n 2.Quadratic Probing ");
        printf("\n 3.Exit ");
        printf("\n Enter your option: ");
        scanf ("%d",&op);
```

```
switch(op) {
    case 1: for (i=0;i<tsize;i++)
        hash[i]=-1 ;
        for(k=0;k<n;k++) {
            key=arr[k] ;
            i = hasht(key);
            while (hash[i]!=-1) {
                i = rehashl(i);
            }
            hash[i]=key ;
        }
        printf("\nThe elements in the array are: ");
        for (i=0;i<tsize;i++) {
            printf("\n Element at position %d: %d",i,hash[i]);
        }
        break;

    case 2: for (i=0;i<tsize;i++)
        hash[i]=-1 ;
        for(k=0;k<n;k++) {
            j=1;
            key=arr[k] ;
            i = hasht(key);
            while (hash[i]!=-1) {
                i = rehashq(i,j);
                j++ ;
            }
            hash[i]=key ;
        }
        printf("\nThe elements in the array are: ");
        for (i=0;i<tsize;i++) {
            printf("\n Element at position %d: %d",i,hash[i]);
        }
        break ;
    }
}while(op!=3);
}
```

SAMPLE OUTPUT

Enter the size of the hash table: 10

Enter the number of elements: 8

Enter Elements: 72 27 36 24 63 81 92 101

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 1

The elements in the array are:
Element at position 0: -1
Element at position 1: 81
Element at position 2: 72
Element at position 3: 63
Element at position 4: 24
Element at position 5: 92
Element at position 6: 36
Element at position 7: 27
Element at position 8: 101
Element at position 9: -1

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 2

The elements in the array are:
Element at position 0: -1
Element at position 1: 81
Element at position 2: 72
Element at position 3: 63
Element at position 4: 24
Element at position 5: 101
Element at position 6: 36
Element at position 7: 27
Element at position 8: 92
Element at position 9: -1

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 3

SAMPLE OUTPUT

Enter the size of the hash table: 7

Enter the number of elements: 7

Enter Elements: 50

700

76

85
92
73
101

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 1

The elements in the array are:
Element at position 0: 700
Element at position 1: 50
Element at position 2: 85
Element at position 3: 92
Element at position 4: 73
Element at position 5: 101
Element at position 6: 76

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 2

The elements in the array are:
Element at position 0: 700
Element at position 1: 50
Element at position 2: 85
Element at position 3: 92
Element at position 4: 73
Element at position 5: 101
Element at position 6: 76

1.Linear Probing
2.Quadratic Probing
3.Exit
Enter your option: 3

Program 3: Develop a menu driven program in C to implement primitive operations on stack a) Push b) Pop c) Display. The program should print appropriate messages for stack overflow and stack underflow

APPLICATIONS / IMPORTANCE:

- 1) Function Call Stack:** In programming, the function call stack is an essential tool for managing function calls, including the application of stack data structure in C. When a function is invoked, its execution context is pushed onto the stack, including local variables and the return address. As the function completes, its stack frame is popped, allowing the program to resume execution in the calling function. This stack-based approach ensures that function calls are handled in the correct order, providing a structured way to manage program flow.
- 2) Undo/Redo Functionality:** Undo and redo functionalities are ubiquitous in software applications, from text editors to graphic design software. Stacks are employed to maintain a history of user actions, allowing for easy reversibility and exploration of past changes. When an action is performed, it's pushed onto the undo stack, and if the user wishes to undo it, the action is popped from the undo stack and pushed onto the redo stack, ensuring a seamless way to navigate through changes and correct mistakes.
- 3) Expression Evaluation:** Expression evaluation in a stack data structure involves using a stack to evaluate arithmetic expressions. When evaluating mathematical expressions, a stack can be employed to keep track of operators and operands, ensuring the correct order of operations. The LIFO property of stacks helps manage operator precedence, allowing for the accurate calculation of complex expressions.
- 4) Backtracking Algorithms:** Backtracking algorithms, such as depth-first search (DFS) in graph theory, depend on the stack data structure to systematically explore paths and make choices at each step. As the algorithm progresses, the current path and choices are pushed onto the stack. If a dead-end is encountered, the algorithm backtracks by popping elements from the stack, allowing it to explore alternative paths and find solutions to problems like path finding and puzzles.
- 5) Browser History:** Web browsers utilize stacks to manage users' browsing history efficiently, demonstrating the application of stack and queue in data structure. Every time a user visits a new web page, the current page is pushed onto the forward stack, while the previous page is pushed onto the back stack. This approach enables users to navigate backwards and forward through their browsing history using the browser's "Back" and "Forward" buttons, providing a seamless and intuitive web browsing experience.

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
#define SIZE 5
int Top=-1, S[SIZE];
void Push();
void Pop();
void Display();
```

```
int main() {
int choice;
while(1) {
    printf("\n Primitive Operations on Stack");
    printf("\n 1.Push the element \n 2.Pop the element \n 3.Display \n 4.Quit");
    printf("\n\n Enter the choice: ");
    scanf("%d",&choice);
    switch(choice) {
    case 1: Push(); break;
    case 2: Pop(); break;
    case 3: Display(); break;
    case 4: exit(0);
    default: printf("\n Invalid choice!!");
    }
    }
}

void Push() {
int x;
if(Top==SIZE-1)
    printf("\n Stack Full. Results in Overflow!!");
else {
    printf("\n Enter element to be inserted to the stack:");
    scanf("%d",&x);
    Top=Top+1;
    S[Top]=x;
    }
}

void Pop() {
if(Top== -1)
    printf("\n Stack Empty. Results in Underflow!!");
else {
    printf("\n Popped element: %d",S[Top]);
    Top=Top-1;
    }
}

void Display() {
if(Top== -1)
    printf("\n No elements in STACK !!");
else {
```

```
printf("\n Elements present in the stack: \n");
for(int i=Top;i>=0;--i)
    printf("\n %d",S[i]);
}
```

SAMPLE OUTPUT

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:10

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:20

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:30

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 3

Elements present in the stack:

30

20

10

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 30

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 3

Elements present in the stack:

20

10

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 20

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:30

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:40

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:50

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 3

Elements present in the stack:

50

40

30

10

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 1

Enter element to be inserted to the stack:60

Primitive Operations on Stack

- 1.Push the element
- 2.Pop the element
- 3.Display
- 4.Quit

Enter the choice: 3

Elements present in the stack:

60

50

40

30

10

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 1

Stack Full. Results in Overflow!!

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 60

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 50

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 40

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 30

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 3

Elements present in the stack:

10

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Popped element: 10

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 2

Stack Empty. Results in Underflow!!

Primitive Operations on Stack

1.Push the element

2.Pop the element

3.Display

4.Quit

Enter the choice: 4

Program 4: Develop a C program to convert INFIX notation to POSTFIX notation**APPLICATIONS / IMPORTANCE:**

- 1) It is useful for expression evaluation and parsing, and is essential for developers working with algorithms and compiler construction.
- 2) Easier to track operator precedence: In postfix expressions, the placement of operators determines the precedence of operators. The operator that occurs first operates on the operand.
- 3) Parenthesis-free expressions: Postfix expressions are parenthesis-free.
- 4) Better understanding of data structures: Implementing the conversion algorithm helps developers understand data structures like stacks.

SOURCE CODE:

```
#include<stdio.h>
#include<ctype.h>

char stack[100];
int top = -1;
void push(char x) {
    stack[++top] = x;
}

char pop() {
    if(top == -1)
        return -1;
    else
        return stack[top--];
}

int priority(char x) {
    if(x == '(')
        return 0;
    if(x == '+' || x == '-')
        return 1;
    if(x == '*' || x == '/')
        return 2;
    return 0;
}

int main() {
```

```
char exp[100];
char *e, x;
printf("Enter valid INFIX expression : ");
scanf("%s",exp);
printf("\n");
e = exp;

printf("\n Equivalent POSTFIX expression : ");

while(*e != '\0') {
    if(isalnum(*e))
        printf("%c ",*e);
    else if(*e == '(')
        push(*e);
    else if(*e == ')') {
        while((x = pop()) != '(')
            printf("%c ", x);
    }
    else {
        while(priority(stack[top]) >= priority(*e))
            printf("%c ",pop());
        push(*e);
    }
    e++;
}

while(top != -1) {
    printf("%c ",pop());
}return 0;
}
```

SAMPLE OUTPUT 1

Enter valid INFIX expression : a+b*c
Equivalent POSTFIX expression : a b c * +

SAMPLE OUTPUT 2

Enter valid INFIX expression : (a+b)*c+(d-a)
Equivalent POSTFIX expression : a b + c * d a - +

Program 5: Develop a C program for evaluation of POSTFIX notation**APPLICATIONS / IMPORTANCE:**

- 1) Its importance stems from the fact that you don't need precedence parentheses when evaluating the expression.
- 2) Since we don't have to deal with things like operator precedence ever — this makes evaluation simple and straightforward.
- 3) In Genetic Algorithms - genetically encoded problem can be performed by using a simple hardware. This particular hardware uses a postfix notation of the fitness expression. Since, in GAs, the same function is evaluated for a fairly large number of times, one need to compile a postfix expression only once at the beginning of the genetic optimization.

SOURCE CODE:

```
#include<stdio.h>
#include<ctype.h>

int stack[20];
int top = -1;

void push(int x) {
    stack[++top] = x;
}

int pop() {
    return stack[top--];
}

int main() {
    char exp[20];
    char *e;
    int n1,n2,n3,num;
    printf("Enter the expression :: ");
    scanf("%s",exp);
    e = exp;
    while(*e != '\0') {
        if(isdigit(*e)) {
            num = *e - 48;
            push(num);
        }
    }
```

```
else    {
    n1 = pop();
    n2 = pop();
    switch(*e)    {
    case '+':
        n3 = n1 + n2;
        break;
    case '-':
        n3 = n2 - n1;
        break;
    case '*':
        n3 = n1 * n2;
        break;
    case '/':
        n3 = n2 / n1;
        break;
    }
    push(n3);
}
e++;
}
printf("\nThe result of expression %s = %d\n\n",exp,pop());
return 0;
}
```

SAMPLE OUTPUT 1

Enter the expression :: 245+*
The result of expression 245+* = 18

SAMPLE OUTPUT 2

Enter the expression :: 567*3/-
The result of expression 567*3/- = -9

Program 6: Develop a menu driven program in C language to implement QUEUE data structures that performs following primitive operations: a) insert b) delete and c) display

APPLICATIONS / IMPORTANCE:

- 1) When a resource is shared among multiple consumers. Examples include CPU scheduling, Disk Scheduling.
- 2) When data is transferred asynchronously (data not necessarily received at same rate as sent) between two processes. Examples include IO Buffers, pipes, file IO, etc.
- 3) Buffers on MP3 players and portable CD players, iPod playlist. Playlist for jukebox - add songs to the end, play from the front of the list.
- 4) When programming a real-time system that can be interrupted (e.g., by a mouse click or wireless connection), it is necessary to attend to the interrupts immediately, before proceeding with the current activity. If the interrupts should be handled in the same order they arrive, then a FIFO queue is the appropriate data structure.
- 5) Simulation of traffic control system

SOURCE CODE:

```
#include<stdio.h>
#include<stdlib.h>
#define N 5

int insert();
int delete();
void display();

int queue[N],ch=1,front=0,rear=0,i,j=1,x=N;

int main() {
    printf("\n Primitive Operations on Queue using Arrays ");
    printf("\n1.Insertion \n2.Deletion \n3.Display \n4.Exit");
    while(ch) {
        printf("\nEnter your Choice :");
        scanf("%d",&ch);
        switch(ch) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: exit(0); break;
        }
    }
}
```

```
}

int insert() {
    if(rear==x)
        printf("\n Queue is Full");
    else {
        printf("\n Enter element to be inserted:");
        scanf("%d",&queue[rear++]);
    }
}

int delete() {
    if(front==rear)
        printf("\n Queue is empty");
    else {
        printf("\n Deleted Element is %d",queue[front++]);
        x++;
    }
}

void display() {
    printf("\nQueue Elements are:\n ");
    if(front==rear)
        printf("\n Queue is Empty");
    else {
        for(i=front; i<rear; i++)
            printf("\t %d",queue[i]);
    }
}
```

SAMPLE OUTPUT

Primitive Operations on Queue using Arrays

1.Insertion

2.Deletion

3.Display

4.Exit

Enter your Choice :1

Enter element to be inserted:10

Enter your Choice :1

Enter element to be inserted:20

Enter your Choice :1

Enter element to be inserted:30

Enter your Choice :3

Queue Elements are:

10 20 30

Enter your Choice :1

Enter element to be inserted:40

Enter your Choice :1

Enter element to be inserted:50

Enter your Choice :3

Queue Elements are:

10 20 30 40 50

Enter your Choice :1

Queue is Full

Enter your Choice :2

Deleted Element is 10

Enter your Choice :2

Deleted Element is 20

Enter your Choice :3

Queue Elements are:

30 40 50

Enter your Choice :2

Deleted Element is 30

Enter your Choice :2

Deleted Element is 40

Enter your Choice :2

Deleted Element is 50

Enter your Choice :3

Queue Elements are:

Queue is Empty

Enter your Choice :2

Queue is empty

Enter your Choice :4

Program 7: Develop a menu driven program in C language to for CIRCULAR QUEUE that performs following primitive operations: a) insert b) delete and c) display

APPLICATIONS / IMPORTANCE

- 1) Regular routine of life is an example of circular queue.
 - Eating - breakfast, lunch, snacks, dinner - breakfast....
 - Days in a week - Sunday - Saturday - Sunday
 - Months in a year - Jan - Dec- Jan ...
- 2) A traffic light
- 3) Luggage carousal in airports.
- 4) Manufacturing units with assembly line for assembling products
- 5) print spooler of operating system

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define SIZE 5

void CQinsert(int);
void CQdelete();
void CQdisplay();

int CQ[SIZE], front = -1, rear = -1;
void main() {
    int choice, value;
    while(1){
        printf("\n***** CIRCULAR QUEUE MENU *****\n");
        printf("1. Insert\n2. Delete\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&choice);
```

```
switch(choice){
    case 1: printf("\nEnter the value to be insert: ");
            scanf("%d",&value);
            CQinsert(value);
            break;
    case 2: CQdelete();
            break;
    case 3: CQdisplay();
            break;
    case 4: exit(0);
    default: printf("\nPlease select the correct choice!!!\n");
}
}
}

void CQinsert(int value) {
    if((front == 0 && rear == SIZE - 1) || (front == rear+1))
        printf("\nCircular Queue is Full! Insertion not possible!!!\n");
    else {
        if(rear == SIZE-1 && front != 0)
            rear = -1;
        CQ[++rear] = value;
        printf("\nInsertion Success!!!\n");
        if(front == -1)
            front = 0;
    }
}

void CQdelete() {
    if(front == -1 && rear == -1)
        printf("\nCircular Queue is Empty! Deletion is not possible!!!\n");
    else {
        printf("\nDeleted element : %d\n",CQ[front++]);
        if(front == SIZE)
            front = 0;
        if(front-1 == rear)
            front = rear = -1;
    }
}

void CQdisplay() {
    if(front == -1)
```

```
    printf("\nCircular Queue is Empty!!!\n");
else {
    int i = front;
    printf("\nCircular Queue Elements are : \n");
    if(front <= rear) {
        while(i <= rear)
            printf("%d\t",CQ[i++]);
    }
    else {
        while(i <= SIZE - 1)
            printf("%d\t", CQ[i++]);
        i = 0;
        while(i <= rear)
            printf("%d\t",CQ[i++]);
    }
}
```

SAMPLE OUTPUT

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 10

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 20

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert

2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 30

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 40

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue Elements are :

10 20 30 40

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 50

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue Elements are :

10 20 30 40 50

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 10

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 20

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue Elements are :

30 40 50

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 60

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display

4. Exit

Enter your choice: 3

Circular Queue Elements are :

30 40 50 60

***** CIRCULAR QUEUE MENU *****

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 2

Deleted element : 30

***** CIRCULAR QUEUE MENU *****

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 3

Circular Queue Elements are :

40 50 60

***** CIRCULAR QUEUE MENU *****

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 1

Enter the value to be insert: 70

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert

2. Delete

3. Display

4. Exit

Enter your choice: 3

Circular Queue Elements are :

40 50 60 70

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 80

Insertion Success!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue Elements are :

40 50 60 70 80

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 1

Enter the value to be insert: 100

Circular Queue is Full! Insertion not possible!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 40

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 50

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 60

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue Elements are :

70 80

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 70

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Deleted element : 80

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 3

Circular Queue is Empty!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 2

Circular Queue is Empty! Deletion is not possible!!!

***** CIRCULAR QUEUE MENU *****

1. Insert
2. Delete
3. Display
4. Exit

Enter your choice: 4

Program 8: Develop a menu driven program in C to perform primitive operations on single linked list.

APPLICATIONS / IMPORTANCE:

- 1) Representation of Polynomials
- 2) Arithmetic Operations on Polynomials
- 3) Addition of Long Positive Integers
- 4) In music players, we can create our song playlist and can play a song either from starting or ending of the list. And these music players are implemented using a linked list.
- 5) We watch the photos on our laptops or PCs, and we can simply see the next or previous images easily. This feature is implemented using a linked list.
- 6) You must be reading this article on your web browser, and in web browsers, we open multiple URLs, and we can easily switch between those URLs using the previous and next buttons because they are connected using a linked list.

SOURCE CODE:

```
#include<stdio.h>
```

```
#include<malloc.h>
```

```
//type declaration of a node
```

```
struct node {
```

```
int data;
```

```
struct node* next;
```

```
};
```

//global head pointer

struct node* head = NULL;

//prototyping of the functions

struct node* create_node(int);

void insert_at_beginning(int);

void insert_at_end(int);

void insert_at_position(int, int);

void delete_at_beginning();

void delete_at_end();

void delete_at_position(int);

void display();

void empty_message(void);

int size_of_list();

int getData();

int getPosition();

int main() {

char ch = 'Y';

int user_choice;

int data, position;

while(ch == 'Y' || ch == 'y') {

printf("\n\n----- Singly Linked List ----- \n");

printf("\n1. Insert a node at beginning");

printf("\n2. Insert a node at end");

printf("\n3. Insert a node at given position");

printf("\n4. Delete a node from beginning");

printf("\n5. Delete a node from end");

printf("\n6. Delete a node from given position");

printf("\n7. Display contents of Linked List ");

printf("\n8. Exit");

printf("\n-----");

printf("\nEnter your choice: ");

```
scanf("%d",&user_choice);

printf("\n-----\n");
switch(user_choice) {
case 1:
printf("\nInserting a node at beginning");
data = getData();
insert_at_beginning(data);
break;

case 2:
printf("\nInserting a node at end");
data = getData();
insert_at_end(data);
break;

case 3:
printf("\nInserting a node at the given position");
data = getData();
position = getPosition();
insert_at_position(data, position);
break;

case 4:
printf("\nDeleting a node from beginning\n");
delete_at_beginning();
break;

case 5:
printf("\nDeleting a node from end\n");
delete_at_end();
break;

case 6:
printf("\nDelete a node from given position\n");
position = getPosition();
delete_at_position(position);
break;

case 7:
printf("\nPrinting the list from beginning\n\n");
```

```
display();
break;

case 8:
printf("\nProgram was terminated\n\n");
return 0;

default:
printf("\n\tInvalid Choice\n");
}

printf("\n.....\n");
printf("\nDo you want to continue? (Y/N) : ");
fflush(stdin);
scanf(" %c", &ch);
}

return 0;
}

void empty_message() {
printf("\n\tList is Empty! ");
}
void memory_message() {
printf("\n Memory can't be allocated ");
}

struct node* create_node(int data) {
struct node* new_node = (struct node*) malloc(sizeof(struct node));

if(new_node == NULL) {
memory_message();
return NULL;
}

new_node->data = data;
new_node->next = NULL;
return new_node;
}
```

/* Insert the new node at the beginning of the list */

```
void insert_at_beginning(int data) {
    struct node* new_node = NULL;
    new_node = create_node(data);

    if(new_node != NULL) {
        new_node->next = head;
        head = new_node;
        printf("\n* Node with data %d was Inserted\n", data);
    }
}
```

/* Insert the new node at the end of the list */

```
void insert_at_end(int data) {
    struct node* new_node = NULL;
    new_node = create_node(data);

    if(new_node != NULL) {
        //if list is empty
        if(head == NULL)
            head = new_node;
        else {
            struct node* last = head;
            //getting the last node
            while(last->next != NULL)
                last = last->next;
            //link the last node next pointer to the new node
            last->next = new_node;
        }
        printf("\n* Node with data %d was Inserted\n", data);
    }
}
```

/* Insert the new node at the given position */

```
void insert_at_position(int data, int pos) {
    //calculate the size of the list
    int list_size = 0;
    list_size = size_of_list();
```

```
//if the list is empty and the position is greater than the 1
if(head == NULL && (pos<= 0 || pos> 1)) {
printf("\nInvalid position to insert a node\n");
return;
}
```

```
// if the list is not empty and the position is out of range
if(head != NULL && (pos<= 0 || pos>list_size)) {
printf("\n Invalid position to insert a node\n");
return;
}
```

```
struct node* new_node = NULL;
new_node = create_node(data);
```

```
if(new_node != NULL) {
struct node* temp = head;
```

```
//getting the position-1 node
int count = 1;
while(count < pos-1) {
temp = temp -> next;
count += 1;
}
```

```
//if the position is 1 then insertion at the beginning
if(pos == 1) {
new_node->next = head;
head = new_node;
}
else {
new_node->next = temp->next;
temp->next = new_node;
}
printf("\n* Node with data %d was Inserted\n", data);
}
}
```

```
/* Delete the node from the beginning of the list */
void delete_at_beginning() {
if(head == NULL) {
```

```
empty_message();
return;
}

struct node* temp = head;
int data = head->data;

    //move head pointer to the next node to the head
head = head->next;
free(temp);

printf("\n* Node with data %d was Deleted\n", data);
}

/* Delete the node from the ending of the list */
void delete_at_end() {
    if(head == NULL) {
        empty_message();
        return;
    }

    struct node* temp = head;
    struct node* prev = NULL;
    int data;

    //reaching the last node
    while(temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }

    data = temp->data;

//if there is only one node
    if(temp == head) {
        free(temp);
        head = NULL;
    }

    else {
        free(temp);
```

```
    prev->next = NULL;
}
printf("\n* Node with data %d was Deleted\n", data);
}
```

/* Deleting the node from the given position */

```
void delete_at_position(int pos) {
    //calculate the size of the list
    int list_size = 0;
    list_size = size_of_list();

    // if the position is out of range
    if(pos <= 0 || pos > list_size) {
        printf("\nInvalid position to delete a node\n");
        return;
    }
    struct node* temp = head;
    struct node* prev = NULL;
    int count = 1;
    while(count < pos) {
        prev = temp;
        temp = temp->next;
        count += 1;
    }
    int data = temp->data;
    if(temp == head) {
        head = head->next;
        free(temp);
    }

    else {
        prev->next = temp->next;
        free(temp);
    }
    printf("\n* Node with data %d was Deleted\n", data);
}
```

/* Prints the data from the start of the list */

```
void display() {
    if(head == NULL) {
        empty_message();
    }
}
```

```
    return;
}

struct node* temp = head;

while(temp != NULL) {
    printf("%d ", temp->data);
    temp = temp->next;
}

/* Getting node data from the user */
int getData() {
    int data;
    printf("\n\nEnter Data: ");
    scanf("%d", &data);

    return data;
}

/* Getting the position of the node from the user */
int getPosition() {
    int pos;
    printf("\nEnter Position: ");
    scanf("%d", &pos);
    return pos;
}

int size_of_list() {
    struct node* temp = head;
    int count = 0;
    while(temp != NULL) {
        count += 1;
        temp = temp->next;
    }
    return count;
}
```

SAMPLE OUTPUT

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 1

Inserting a node at beginning

Enter Data: 10

* Node with data 10 was Inserted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 1

Inserting a node at beginning

Enter Data: 20

* Node with data 20 was Inserted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning

20 10

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 2

Inserting a node at end

Enter Data: 30

* Node with data 30 was Inserted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning

20 10 30

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 3

Inserting a node at the given position

Enter Data: 40

Enter Position: 2

* Node with data 40 was Inserted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning
20 40 10 30

.....
Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 4

Deleting a node from beginning
* Node with data 20 was Deleted

.....
Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning

40 10 30

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 5

Deleting a node from end

* Node with data 30 was Deleted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end

3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning

40 10

.....
Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 1

Inserting a node at beginning

Enter Data: 70

* Node with data 70 was Inserted

.....
Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning

5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 7

Printing the list from beginning

70 40 10

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position
7. Display contents of Linked List
8. Exit

Enter your choice: 6

Delete a node from given position

Enter Position: 2

* Node with data 40 was Deleted

.....

Do you want to continue? (Y/N) : y

----- Singly Linked List -----

1. Insert a node at beginning
2. Insert a node at end
3. Insert a node at given position
4. Delete a node from beginning
5. Delete a node from end
6. Delete a node from given position

7. Display contents of Linked List

8. Exit

Enter your choice: 7

Printing the list from beginning

70 10

.....

Do you want to continue? (Y/N) : n

Program 9: Write a C program to perform primitive operations (insertion, deletion and traversal) on Binary Search Tree

APPLICATIONS / IMPORTANCE:

1. Searching and Retrieval: BST trees are mainly utilized for effective data retrieval and searching operations. The binary search property helps to guarantee that the search operation can be completed in $O(\log n)$ time, where n is the number of nodes in the tree.

2. Database Systems: Binary search trees are used in various databases to search and index large, scattered reports. For example, we can store the names using the BST tree structure in the phonebook.

3. Auto-Complete and Spell Check: A binary search tree can implement auto-complete functionality at various places, like search engines. It can quickly suggest completions while typing based on the prefix entered. They are also used in spell-checking algorithms to suggest corrections for incorrectly spelled words.

4. File Systems: Various current version file systems use the binary search algorithm to store files in directories.

5. Priority Queues: They can also be used to implement priority queues. The key of each element represents its priority, and you can efficiently extract the element with the highest (or lowest) priority.

6. Optimization Problems: Binary search trees can be used to solve various optimization problems. For instance, in the field of dynamic programming, BSTs can be used to find the optimal solution for specific problems efficiently.

7. Implement Decision Tree: It can implement decision trees in artificial intelligence and machine learning algorithms. It is used to predict outcomes and decisions of the models. These trees can help in various fields like diagnosis, research, and financial analysis work.

8. Encryption Algorithms: It can help encrypt sensitive information using a key encryption algorithm by generating public and private keys.

9. Compressing Data: It can also help in compressing extensive data and optimizing space. It can help implement various applications like image, file, audio, video, etc.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct btnode {
    int value;
    struct btnode *l;
    struct btnode *r;
}*root = NULL, *temp = NULL, *t2, *t1;

void delete1();
void insert();
void delete();
void inorder(struct btnode *t);
void create();
void search(struct btnode *t);
void preorder(struct btnode *t);
void postorder(struct btnode *t);
void search1(struct btnode *t,int data);
int smallest(struct btnode *t);
int largest(struct btnode *t);

int flag = 1;

void main() {
    int ch;

    printf("\nOPERATIONS ---");
    printf("\n1 - Insert an element into tree\n");
    printf("\n2 - Delete an element from the tree\n");
    printf("\n3 - Inorder Traversal\n");
    printf("\n4 - Preorder Traversal\n");
    printf("\n5 - Postorder Traversal\n");
    printf("\n6 - Exit\n");
    while(1) {
        printf("\nEnter your choice : ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
```

```
        insert();
        break;
    case 2:
        delete();
        break;
    case 3:
        inorder(root);
        break;
    case 4:
        preorder(root);
        break;
    case 5:
        postorder(root);
        break;
    case 6:
        exit(0);
    default :
        printf("Wrong choice, Please enter correct choice ");
        break;
    }
}
```

```
/* To insert a node in the tree */
void insert() {
    create();
    if (root == NULL)
        root = temp;
    else
        search(root);
}
```

```
/* To create a node */
void create() {
    int data;

    printf("Enter data of node to be inserted : ");
    scanf("%d", &data);
    temp = (struct btnode *)malloc(1*sizeof(struct btnode));
    temp->value = data;
    temp->l = temp->r = NULL;
```

```
}
```

```
/* Function to search the appropriate position to insert the new node */
```

```
void search(struct bnode *t){  
    if ((temp->value > t->value) && (t->r != NULL))    /* value more than root node value  
    insert at right */  
        search(t->r);  
    else if ((temp->value > t->value) && (t->r == NULL))  
        t->r = temp;  
    else if ((temp->value < t->value) && (t->l != NULL))    /* value less than root node value  
    insert at left */  
        search(t->l);  
    else if ((temp->value < t->value) && (t->l == NULL))  
        t->l = temp;  
}
```

```
/* recursive function to perform inorder traversal of tree */
```

```
void inorder(struct bnode *t) {  
    if (root == NULL)    {  
        printf("No elements in a tree to display");  
        return;  
    }  
    if (t->l != NULL)  
        inorder(t->l);  
    printf("%d -> ", t->value);  
    if (t->r != NULL)  
        inorder(t->r);  
}
```

```
/* To check for the deleted node */
```

```
void delete() {  
    int data;  
  
    if (root == NULL)    {  
        printf("No elements in a tree to delete");  
        return;  
    }  
    printf("Enter the data to be deleted : ");  
    scanf("%d", &data);  
    t1 = root;  
    t2 = root;
```

```
    search1(root, data);
}

/* To find the preorder traversal */
void preorder(struct btnode *t) {
    if (root == NULL) {
        printf("No elements in a tree to display");
        return;
    }
    printf("%d -> ", t->value);
    if (t->l != NULL)
        preorder(t->l);
    if (t->r != NULL)
        preorder(t->r);
}

/* To find the postorder traversal */
void postorder(struct btnode *t) {
    if (root == NULL) {
        printf("No elements in a tree to display ");
        return;
    }
    if (t->l != NULL)
        postorder(t->l);
    if (t->r != NULL)
        postorder(t->r);
    printf("%d -> ", t->value);
}

/* Search for the appropriate position to insert the new node */
void search1(struct btnode *t, int data) {
    if ((data > t->value)) {
        t1 = t;
        search1(t->r, data);
    }
    else if ((data < t->value)) {
        t1 = t;
        search1(t->l, data);
    }
    else if ((data == t->value)) {
        delete1(t);
    }
}
```

```
    }
}

/* To delete a node */
void delete1(struct btnode *t) {
    int k;
    /* To delete leaf node */
    if ((t->l == NULL) && (t->r == NULL)) {
        if (t1->l == t) {
            t1->l = NULL;
        }
        else {
            t1->r = NULL;
        }
        t = NULL;
        free(t);
        return;
    }

    /* To delete node having one left hand child */
    else if ((t->r == NULL)) {
        if (t1 == t) {
            root = t->l;
            t1 = root;
        }
        else if (t1->l == t) {
            t1->l = t->l;
        }
        else {
            t1->r = t->l;
        }
        t = NULL;
        free(t);
        return;
    }

    /* To delete node having right hand child */
    else if (t->l == NULL) {
        if (t1 == t) {
            root = t->r;
            t1 = root;
        }
    }
}
```

```
else if (t1->r == t)
    t1->r = t->r;
else
    t1->l = t->r;
t == NULL;
free(t);
return;
}
```

```
/* To delete node having two child */
else if ((t->l != NULL) && (t->r != NULL)) {
    t2 = root;
    if (t->r != NULL) {
        k = smallest(t->r);
        flag = 1;
    }
    else {
        k = largest(t->l);
        flag = 2;
    }
    search1(root, k);
    t->value = k;
}
}
```

```
/* To find the smallest element in the right sub tree */
int smallest(struct btnode *t) {
    t2 = t;
    if (t->l != NULL) {
        t2 = t;
        return(smallest(t->l));
    }
    else
        return (t->value);
}
```

```
/* To find the largest element in the left sub tree */
int largest(struct btnode *t) {
    if (t->r != NULL) {
        t2 = t;
        return(largest(t->r));
    }
}
```

```
else
    return(t->value);
}
```

SAMPLE OUTPUT:**OPERATIONS ---**

- 1 - Insert an element into tree
- 2 - Delete an element from the tree
- 3 - Inorder Traversal
- 4 - Preorder Traversal
- 5 - Postorder Traversal
- 6 - Exit

Enter your choice : 1
Enter data of node to be inserted : 50

Enter your choice : 1
Enter data of node to be inserted : 40

Enter your choice : 1
Enter data of node to be inserted : 60

Enter your choice : 1
Enter data of node to be inserted : 45

Enter your choice : 1
Enter data of node to be inserted : 65

Enter your choice : 1
Enter data of node to be inserted : 55

Enter your choice : 1
Enter data of node to be inserted : 30

Enter your choice : 3
30 -> 40 -> 45 -> 50 -> 55 -> 60 -> 65 ->
Enter your choice : 4
50 -> 40 -> 30 -> 45 -> 60 -> 55 -> 65 ->

Enter your choice : 5
30 -> 45 -> 40 -> 55 -> 65 -> 60 -> 50 ->
Enter your choice : 2
Enter the data to be deleted : 50

Enter your choice : 3
30 -> 40 -> 45 -> 55 -> 60 -> 65 ->
Enter your choice : 4
55 -> 40 -> 30 -> 45 -> 60 -> 65 ->
Enter your choice : 5
30 -> 45 -> 40 -> 65 -> 60 -> 55 ->
Enter your choice : 4
55 -> 40 -> 30 -> 45 -> 60 -> 65 ->
Enter your choice : 6

Program 10: Develop a C program to traverse a graph using Breadth First Search

APPLICATIONS / IMPORTANCE:

- 1) **Shortest Path in Unweighted Graph:** One of the most common applications of BFS is finding the shortest path in an unweighted graph. Since BFS explores all nodes at the current depth before moving to the next level it guarantees the shortest path from the source to any other node in terms of the number of edges. This makes BFS highly efficient for path finding problems where edge weights are uniform or irrelevant.
- 2) **Social Network Analysis:** In social network analysis BFS helps us understand how people are connected to each other. It can find the shortest link between two people and also helps in finding groups or communities in a network. For example it can show how many steps it takes to reach a friend of a friend. This is one of the common applications of BFS in apps like Facebook LinkedIn or Instagram.
- 3) **Web Crawlers:** Web crawlers use BFS to visit pages on the internet. The crawler starts from one page and visits all nearby links first before going deeper. This helps in collecting and indexing web pages in a proper order. Search engines like Google use this method to scan websites. This makes BFS very useful in building search tools.
- 4) **Level Order Traversal in Trees:** BFS is also used in level order traversal of trees. It means visiting all the nodes of the tree level by level from top to bottom. This is helpful in printing the tree structure or solving tree based problems. It is one of the simple and clear applications of BFS in data structures and programming.
- 5) **Cycle Detection in Unweighted Graph:** Another helpful application of BFS is cycle detection in unweighted graphs. When we use BFS to visit nodes we can keep track of the parent of each node. If we find a node that was already visited and it is not the

parent then a cycle exists. This is useful in checking whether a graph is safe to use or not especially in computer networks and scheduling tasks.

- 6) **Network Broadcasting:** BFS is useful in network broadcasting. It helps send a message or signal from one computer to all other computers in a network. BFS makes sure the message travels step by step and reaches all parts of the network quickly. This is very useful in real time systems and communication networks where fast message delivery is needed.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int queue[MAX], front = -1, rear = -1;
int visited[MAX] = {0};

void enqueue(int v) {
    if (rear == MAX - 1) return;
    if (front == -1) front = 0;
    queue[++rear] = v;
}

int dequeue() {
    if (front == -1) return -1;
    int v = queue[front++];
    if (front > rear) front = rear = -1;
    return v;
}

void BFS(int graph[MAX][MAX], int start, int n) {
    enqueue(start);
    visited[start] = 1;
    printf("BFS Traversal: ");

    while (front != -1) {
        int v = dequeue();
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (graph[v][i] && !visited[i]) {
                enqueue(i);
                visited[i] = 1;
            }
        }
    }
}
```

```
int main() {
    int graph[MAX][MAX], n, start;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    printf("Enter the adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("Enter the starting vertex: ");
    scanf("%d", &start);
    BFS(graph, start, n);
    return 0;
}
```

SAMPLE OUTPUT:

```
Enter the number of vertices: 6
Enter the adjacency matrix:
0 1 1 0 0 0
0 0 1 1 0 0
0 0 0 1 0 0
0 0 0 0 1 0
1 1 0 0 0 1
0 0 0 0 0 0
Enter the starting vertex: 0
BFS Traversal: 0 1 2 3 4 5
```

SAMPLE VIVA QUESTIONS

1	What is data structure?
2	Differentiate between file and structure storage structure.
3	When is a binary search best applied?
4	What is a linked list?
5	How do you reference all the elements in a one-dimension array?
6	In what areas do data structures are applied?
7	What is LIFO?
8	What is a queue?
9	What are binary trees?
10	Which data structures are applied when dealing with a recursive function?
11	What is a stack?
12	Explain Binary Search Tree
13	What are multidimensional arrays?
14	Are linked lists considered linear or non-linear data structures?
15	How does dynamic memory allocation help in managing data?
16	What is FIFO?
17	What is an ordered list?
18	What is merge sort?
19	Differentiate NULL and VOID
20	What is the primary advantage of a linked list?
21	What is the difference between a PUSH and a POP?
22	What is a linear search?
23	How does variable declaration affect memory allocation?
24	What is the advantage of the heap over a stack?
25	What is a postfix expression?
26	What is Data abstraction?
27	How do you insert a new item in a binary search tree?
28	How does a selection sort work for an array?
29	How do signed and unsigned numbers affect memory?
30	What is the minimum number of nodes that a binary tree can have?
31	What are dynamic data structures?
32	In what data structures are pointers applied?

33	Do all declaration statements result in a fixed reservation in memory?
34	What are ARRAYS?
35	What is the minimum number of queues needed when implementing a priority queue?
36	Which sorting algorithm is considered the fastest?
37	Differentiate STACK from ARRAY.
38	Give a basic algorithm for searching a binary search tree.
39	What is a dequeue?
40	What is a bubble sort and how do you perform it?
41	What is a Data Structure?
42	Describe the types of Data Structures?
43	What is a Linear Data Structure? Name a few examples.
44	What are some applications of Data Structures?
45	What is the difference between file structure and storage structure?
46	What is a multidimensional array?
47	How are the elements of a 2D array stored in the memory?
48	What is an algorithm?
49	Why do we need to do an algorithm analysis?
50	Which sorting algorithm is considered the fastest? Why?
51	What is an asymptotic analysis of an algorithm?
52	What are asymptotic notations?
53	List the types of trees?
54	Differentiate NULL and VOID
55	What is data abstraction?
56	What is Huffman's algorithm?
57	How many queues do you need to execute a priority queue?
58	What are the applications of graph Data Structure?
59	What are recursive algorithms?
60	What is the use of void pointers?